IntoToArduino_Student.odt
17dec25
25jun26

Introduction to Using the Arduino Development System

ref1: https://docs.arduino.cc/hardware/uno-rev3/

Student Notes

# Table of Contents



## 0. Setting up the Arduino IDE

Go to the Arduino website at

https://www.arduino.cc/ > For Makers > Go to Documentation > Software > Arduino IDE 2

There is a walk-through for installing the Arduino IDE and hardware plus lots of other valuable information about the Arduino product line, project ideas, and additional help.

## 1. Blink; the hardware “Hello World” application.

This program can be found in

File > Examples > 01.Basics > Blink

Click Blink to load the program into the IDE. Connect your Arduino board to the computer using the USB cable.

In the pull-down “Select Board” in the menu bar, select the board connected if the system has not already set this for you.
This program is stored in the installation folders and these should not be changed. So the first thing to do, if you want change this program is to save into your own folder. Use File > Save As… and select the path for your folder. In the folder, unclick the read-only file property of the Blink.ino file and load the file from your folder into the IDE for editing. I use “C:\Users\<your name>\Documents\Arduino” for my files.
On line 34, change the delay(1000) to delay(100). Click the Upload arrow and the LED should now Blink quickly each second. This shows that you have control of the hardware though your programming.

## 2. LEDs

Some electronics theory is now needed so that parts (the Arduino or the LEDs) are not damaged.
The first question to be answered is “What does it take to light up an LED?”. The standard red LED requires between 1.8v and 2.2v across it and at least 1ma of current available. The schematic symbol and physical case are shown in Figure 1. The FLAT side of the circular LED case indicates the Cathode or negative terminal.

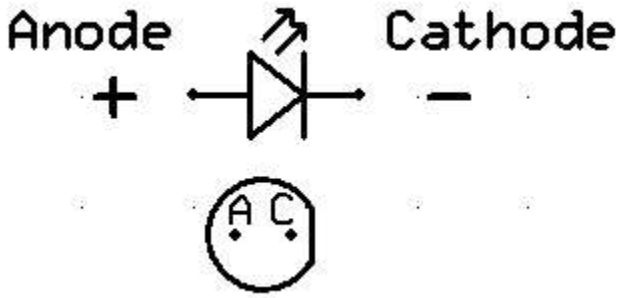


Figure 1

The Anode is connected to the positive voltage source and the Cathode is connected to the negative or ground side. Current (by convention) flows from positive to negative.
At 1ma., it will be very dim, but will light up. Most LEDs can handle 20ma and up to 30ma for a short duration. We will assume that the LED drops 2v and will use 10ma for brightness. Brightness can be controlled by how much current is used, but the voltage drop across the LED will not change.
Looking at the Arduino UNO 3 hardware specifications (see ref1 Tech Specs), the Arduino I/O pins can supply 5V @ 20ma.
The problem then is provide the correct voltage to the LED at a desired current. The solution is to use a resistor in series with the LED and Ohm’s Law to calculate the resistor’s value.

This is a simple circuit for the Arduino connected to an LED.

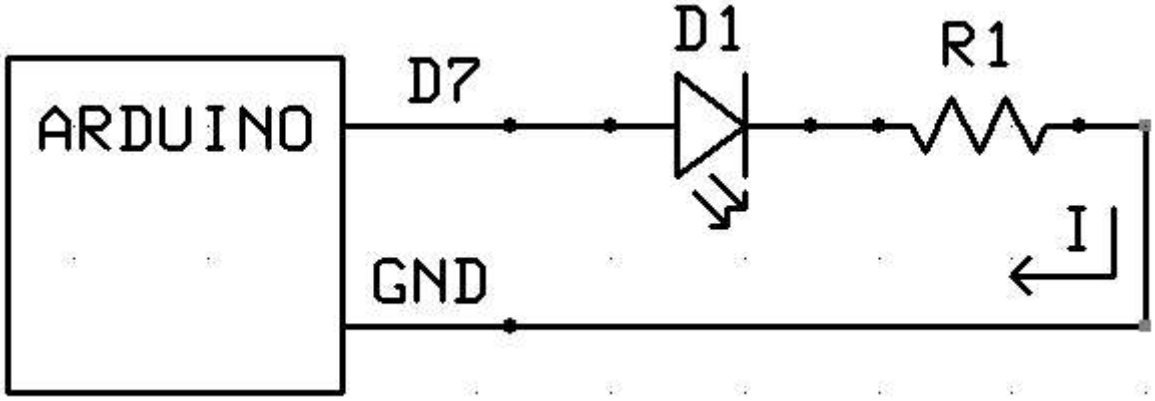


Figure 2

It is a ‘series’ circuit because all of the components are strung together in a series, one after another. There’s another law called Kirchoff’s Voltage Law (KVL) that states that “The sum of the voltages (drops and rises) in a series circuit must add up to zero.” Another Kirchoff Law states the “Current entering a node equals the current exiting a node.” In a series circuit, each connection is a node with one input and one output. Therefore all of the connection have the same current (I) flowing though it and each component has the same current flowing though it. (i.e. the current is the same everywhere in the circuit).

Following the circuit in Figure 2, the voltage starts a +5v (Arduino I/O Pin), then drops 2v across the LED, then drops Vr across the resistor, and ends up at 0v and Ground (GND). Using KVL, 5 – 2 – 3 = 0, so the resistor must drop 3 volts. Using Ohm’s Law that E = IR or Voltage = Current x Resistance, the known values are plugged into the equation. $V_r = I \times R$. $V_r$ and I are know. R is to be solved for. Rearrange the equation to solve for R.

$V_r/I = R \rightarrow 3v / 10ma = 3v / 0.01A = 300$ ohms.

The nice thing about LEDs is that their brightness is subjective; a little dimmer or a little brighter is hardly noticeable. If a 300 ohm resistor is not available, use a 270 ohm (11ma) or a 330 ohm (9ma) resistor. These are common values for resistors. Other common values are 150, 220, 390, and 470. Any of these will cause the LED to light up.

### 3. Adding external LEDs to the Arduino aboard.

Apply information from #2 to preform experiments with six LEDs. Connect up to six LEDs to the Arduino board as shown in Figure 3. Different resistor values may be used based on availability or desired brightness.

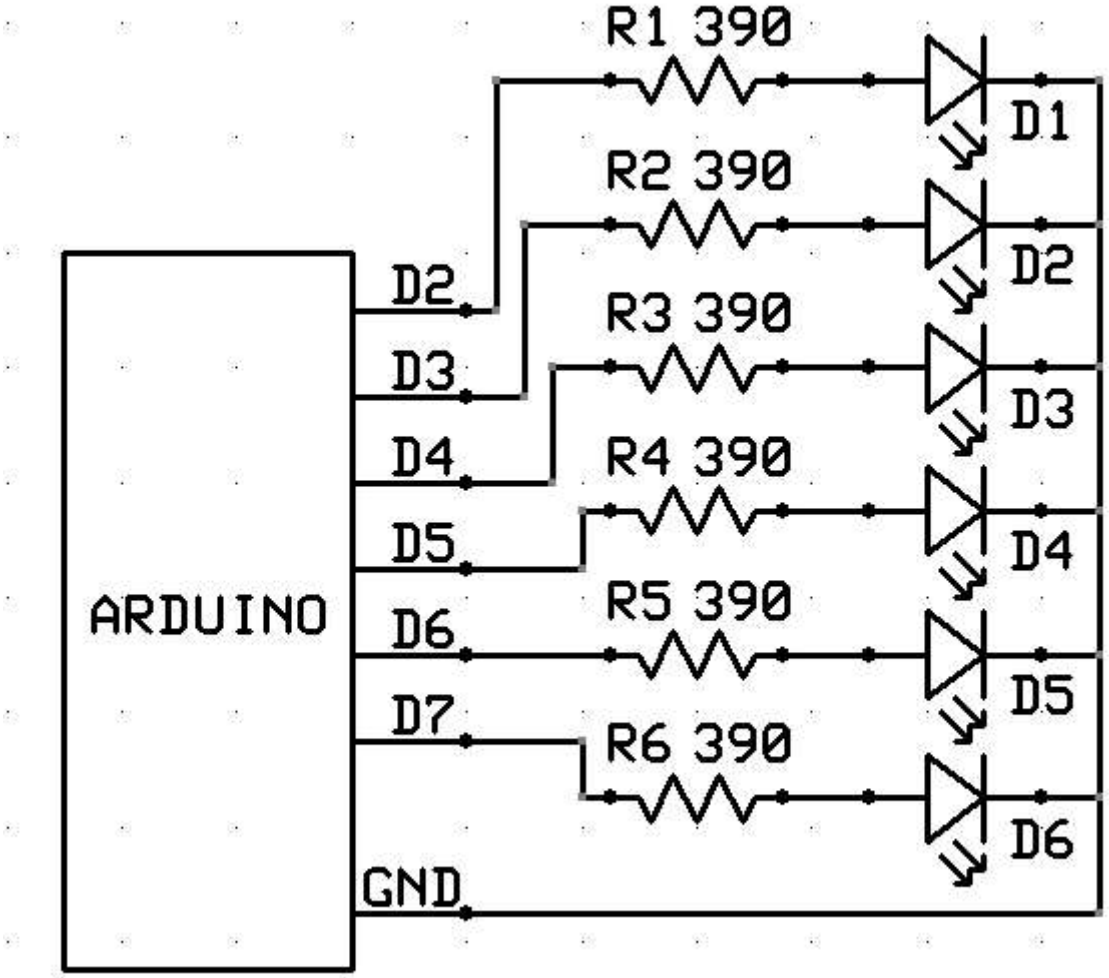


Figure 3

3a. Turn each LED ON; one at a time, then turn each LED OFF, one at a time and repeat.

In programming, there are many ways to do the same thing. The follow section will explore this concept.

First though, a few words on how the Arduino processes code. Simple projects can be written on 'C'. C++ and Python are also available.
The Arduino starts up by running it's Bootloader. This block of code contains the support functions, timing, configuration code, etc. Once the Bootloader completes initialization, it calls the setup() function in the User sketch (code). The setup() function is expected to return back to the Bootloader. Any value returned is ignored. The Bootloader will then call the User loop() function, which is expected to return back to the Bootloader in a reasonable amount of time, say less than one second. It then calls loop() again. The call/return process continues until the User sketch decides to stop.
In summary, setup() runs once at the beginning of the sketch and loop() is called over and over again.

## 3a1. Straight line code.

It's usually easiest to start with a straight-line code approach to any new problem. This can provide insight into how the hardware controls an external device and logical issues of control.
Using the Blink sketch as a reference, a few things need to be added to control more LEDs connected to the I/O pins.
Use File > Open > the LED_3a1 file, located in the ITA_Examples folder provided, to load the example code into the IDE.

Reading though the sketch, the first section gives a name to each 'magic' number used by the hardware control and configuration functions. LED2 is more informative than 3.
The next section sets up the I/O lines to drive the LEDs. Look up these functions in the Reference for more information.
The last section is the loop() function. Each time it is called, it steps through the process of turning ON/OFF each LED using a 100ms delay between each change.

Possible Modifications
1. Change the sketch to turn each LED, with a small delay between each, and then turn each LED OFF, with a small delay between each.
2. Change the sequence order of which LED are turned ON or OFF.

## 3a2. Using an **Array** to control which LED is Blinked.

Use File > Open > the LED_3a2 file.

This sketch results in the same display as the 3a1 sketch, but in a different manner.
It places all of the LED references into an array and then steps though the array extracting the LED references one at a time using a for() loop control structure.
Using a for() loop for a process that does the same operation for different values can reduce errors or cause then to show up earlier since the same code is used for all values.

The array has been added to the first section.
The setup() code is reduced to a for() loop the walks through the array to set up the I/O.
The loop() code is also reduced. It's process has been changed to do one LED blink per loop() call.
The index variable is 'static' so that it retains its value between loop() calls. This variable could have been placed above the setup() to make it a global variable which would give the same results.
(see Help > Reference > Variable Scope & Qualifiers for more information)

Possible Modifications
1. Add more LEDs to the array (from the available LEDs) to increase the display.

2. Change the delay() times to make the display cycle faster or slower.

3a3. Using an Array of **Data Structures** {LED#, State, time} to control output and delay.

Use the Array as a circular pattern generator.
Use the Array as a scrolling patter generator; Chase, Ping-pong, Morse Code.

3a4. Using the random function to randomly pick an LED from the Array to turn ON/OFF or Toggle.
Use an Array to record the **State** of an LED.

3b. Use a **State Machine** to cycle through different display types.
3b1. Use time checks to set the run-time for each state.
3b2. Use a **Table** to control the state sequence.

## 4. 7 Segment LEDs

### 4a. Single 7-Segment LED Display

#### 4a1. Using Arduino functions – sketch LED7seg_4a1.ino

A 7-segment LED is just 8 LEDs arranged to display characters. The anode of the LED is attached to it's respective letter pin (a, b, c, etc.) and all LED cathodes are attached to both of the C (common) pins 3 and 8. So, the connections are the same as in Section3, except for eight LED instead of 6. The other difference in the hardware is the use of a transistor to sink the current from all of the LEDs. An Arduino pin can only sink 20ma. If all of the LEDs are ON, then the common line could have as much as 160ma which is beyond the Arduino's capability. Therefore, a NPN transistor is used.

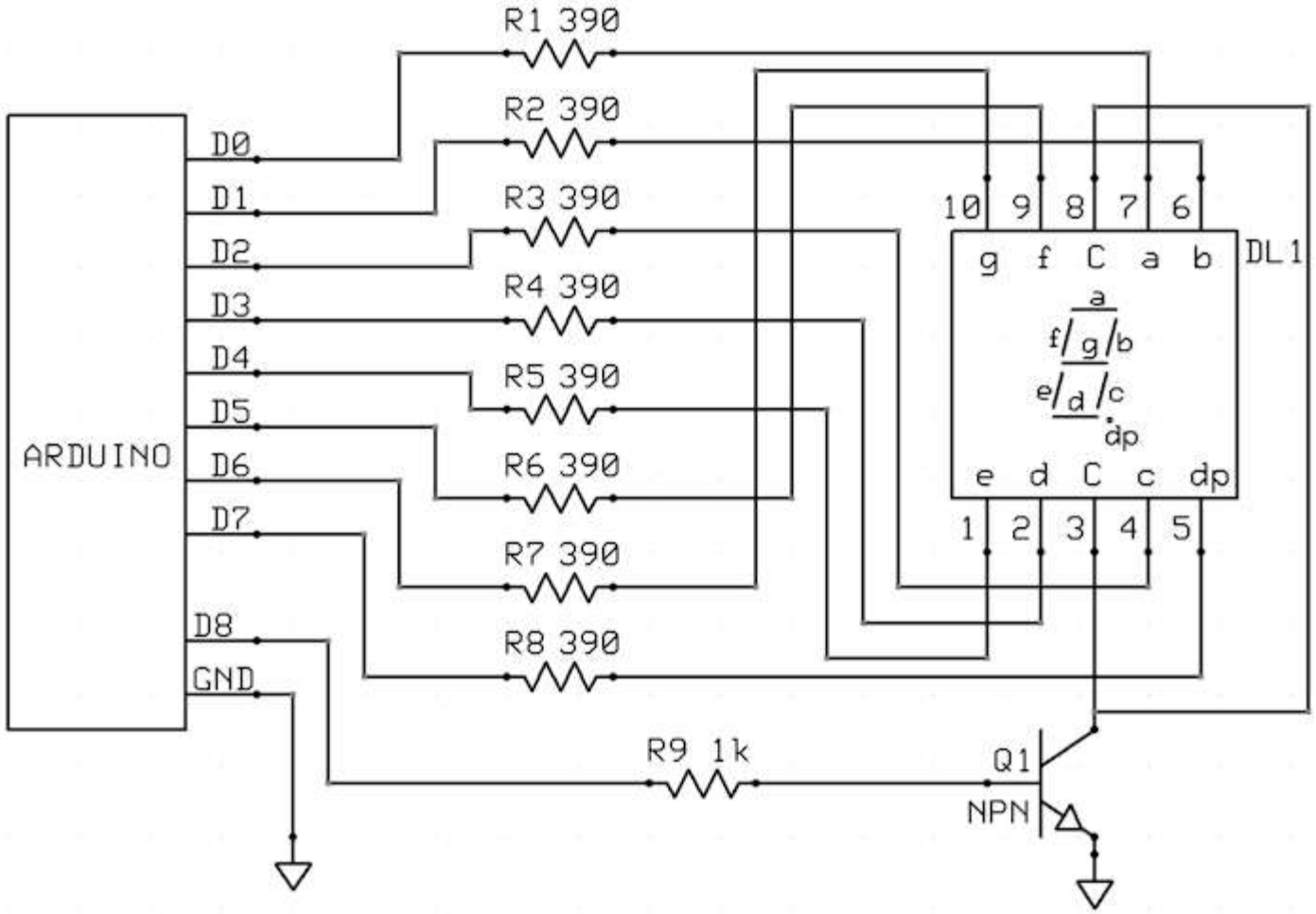


Figure 4

4a1a. Removing Delay() calls.

4a2. Using direct hardware control registers

4b. Multiple 7-Segment LED Displays

## 5. Reading Switches and Key Pads

There are two basic types of switches that are connected to an Arduino: Dynamic and Static.
Dynamic switches are usually momentary contact, like push button, micro-switch, optical, etc. The issue with these types is that they can generate noise or 'switch bounce' when they close. This noise can be mitigated with timing logic in the code.
Static switches are, well, static for more of their use. These are toggle switches, dip switches, rotary switches, etc. They are usually set once and forgotten. Code for these is very simple.
A switch connected to a digital line configured as an INPUT can be in one of two configurations: Pull-up or Pull-down. (see Figure X)
In a Pull-up configuration, the input pin is pulled up to +5v (a logical HIGH) when the switch is OPEN and pulled down to GROUND (a logical LOW) when the switch is closed. This is the most common configuration. The digital pin configuration function, pinMode(), has a mode INPUT_PULLUP option that provides an internal pull-up resistor to remove the need for an external one.
In a Pull-down configuration, the input pin is pulled down to GROUND (a logical LOW) when the switch is OPEN and pulled up to +5v (a logical HIGH) when the switch is closed. This is rare, but might be needed for custom

pre-built hardware. Some times a protection resistor is added in serial to prevent shorting the pin to +5v if it was accidentally configures as an OUTPUT and left in a LOW condition.

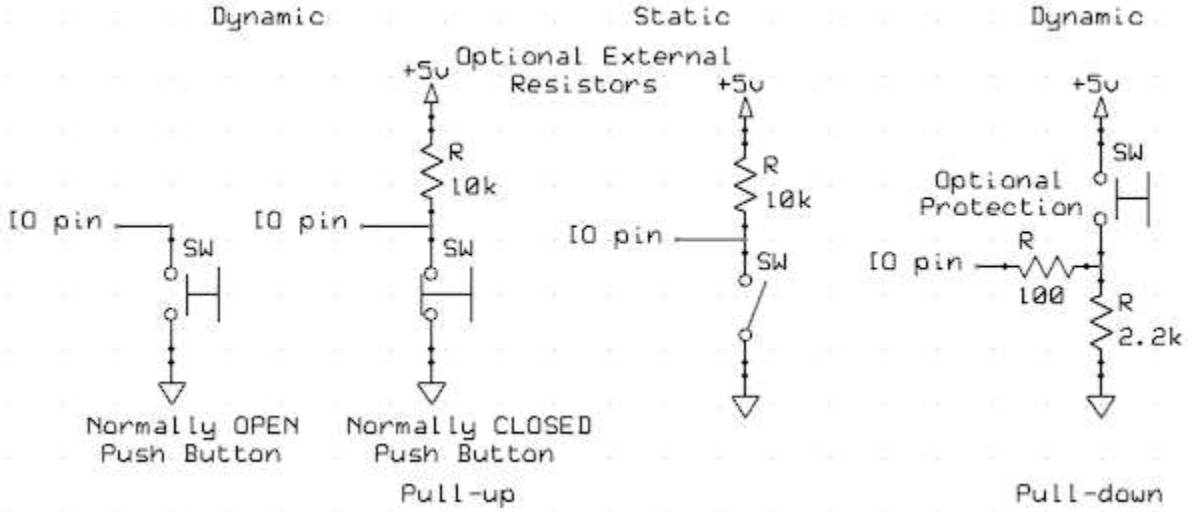


Figure 5.

## 5a. Reading a Static Switch

Reading the state (ON or OFF) of a static switch is simple. In the setup() function, configure the pin to be used as an input with the internal pull-up resistor active using the pinMode() function. In this example, the built in LED on pin 13 will be used to indicate the state of the switch.

```
void setup() {
 pinMode(5, INPUT_PULLUP);    // use pin 5 as a INPUT. Activate internal resistor.
 pinMode(13, OUTPUT);         // configure to use on-board LED.
}

// put your main code here, to run repeatedly:
void loop() {
#if 1
 // Simple way.
 if(digitalRead(5) == LOW) {
  digitalWrite(13, LOW);         // Turn OFF LED.
 } else {
  digitalWrite(13, HIGH);        // Turn ON LED.
 }
#else
 // Compact way
 digitalWrite(13, digitalRead(5));
#endif
}
```

This example code is in ../ITA_Examples/Switch_5a.

## 5b. Reading a Dynamic Switch

Momentary contact switched like push button and micro-switches may not make solid contact when they first close and can generate a 'ringing' noise of closed, open, closed, etc. for a few milliseconds. This is call 'switch bounce' due to the mechanical contacts not making a solid connection at first.
A 'Switch debounce' routine is used to filter out this bounce by inserting wait delay until the switch is tested again.
In general, the method returns TRUE if the switch is pressed down and FALSE when the switch has been released. In both cases, a flag is set to indicate that the switch state is valid. The logic flow is shown below. It can be implemented using a time comparison with millis().

### 5b1. Simple code method – can spend 10ms or more in function before returning.

```
bool isSwitchClosed()
if not waiting for switch to be released
    if switch is pressed
        bounceDelay = millis() + 10
        while( bounceDelay > millis() )        // wait until switch is continuously pressed for 10ms.
            if switch is NOT still pressed
                bounceDelay = millis() + 10
        set waitForHIgh flag = true
        return true
else
    // Keep true until switch is released.
    if switch is released
        set waitForHigh flag = false
    else
        return true
```

Example code is in ../ITA_Examples/Switch_5b1.

### 5b2. Complex code method – Does not waste time in function. Returns immediate state. Called each loop.

```
bool isSwitchClosed()
{
  static bool timingLow = false;
  static bool waitForHigh = false;
  bool results = false;                          // default value

  // Waiting for the switch to be released?
  if( !waitForHigh ) {
    if( timingLow ) {
      if( bounceDelay < millis() ) {
        timingLow = false;                         // bounce time past.
        waitForHigh = true;                        // don't test again until released.
      }
      results = true;                              // still pressed.
    } else {
      // normal test
      if( digitalRead(PUSH_SWITCH) == LOW ) {        // if switch is pressed
        bounceDelay = millis() + 10;                 // set to wait 10ms.
        timingLow = true;
        results = true;                              // switch is closed.
      }
    }
  } else {
    // Yes. Test if released.
```

```
      if( digitalRead(PUSH_SWITCH) == HIGH ) {    // release. clear flag.
        waitForHigh = false;
      } else {
        results = true;                           // still pressed.
      }
  }
  return results;
}
```

Example code is in ../ITA_Examples/Switch_5b2.
Example code Switch_5b3 in ../ITA_Examples show counters counting close and open events. It demonstrates that a debounce on release may also be needed.

### 5b3. Complex code method – with print

### 5b4. Complex code method – Using a State Machine.

Does not waste time in function. Returns immediate state. Called each loop. Eliminates bounce on contact and release.
TODO: Logic flow and example code.

### 5c. KeyPad Interface

Interfacing to a 4x4 keypad uses the mechanics discussed in section 5b with the addition of have to scan the rows and columns for key presses. One issue with membrane keypads (the most common kind) is that they are slightly capacitive and the system has to wait a bit after activating the row to be read. The set up is shown in Figure 5c.

There are two basic approaches to a keypad scan. 1. Return only one key that is pressed or 2. Return ALL keys that were pressed during the scan. Both methods will return the key press information in an unsigned int of 16 bits with this encoding.

R1C1 R1C2 R1C3 R1C4 R2C1 R2C2 R2C3 R2C4 R3C1 R3C2 R3C3 R3C4 R4C1 R4C2 R4C3 R4C4

A ‘1’ in the bit position indicates that the switch is closed.
Both methods will use a simplified debounce process. If the scan pattern does not change for 10ms, it is encoded and returned. When ALL keys are released for 10ms, a zero pattern will be returned. The zero pattern will continue to be returned until a valid scan is generated.
A more complicated system that lets the user ‘walk across’ the key pad, returning the last key press, is left as an exercise for the student.

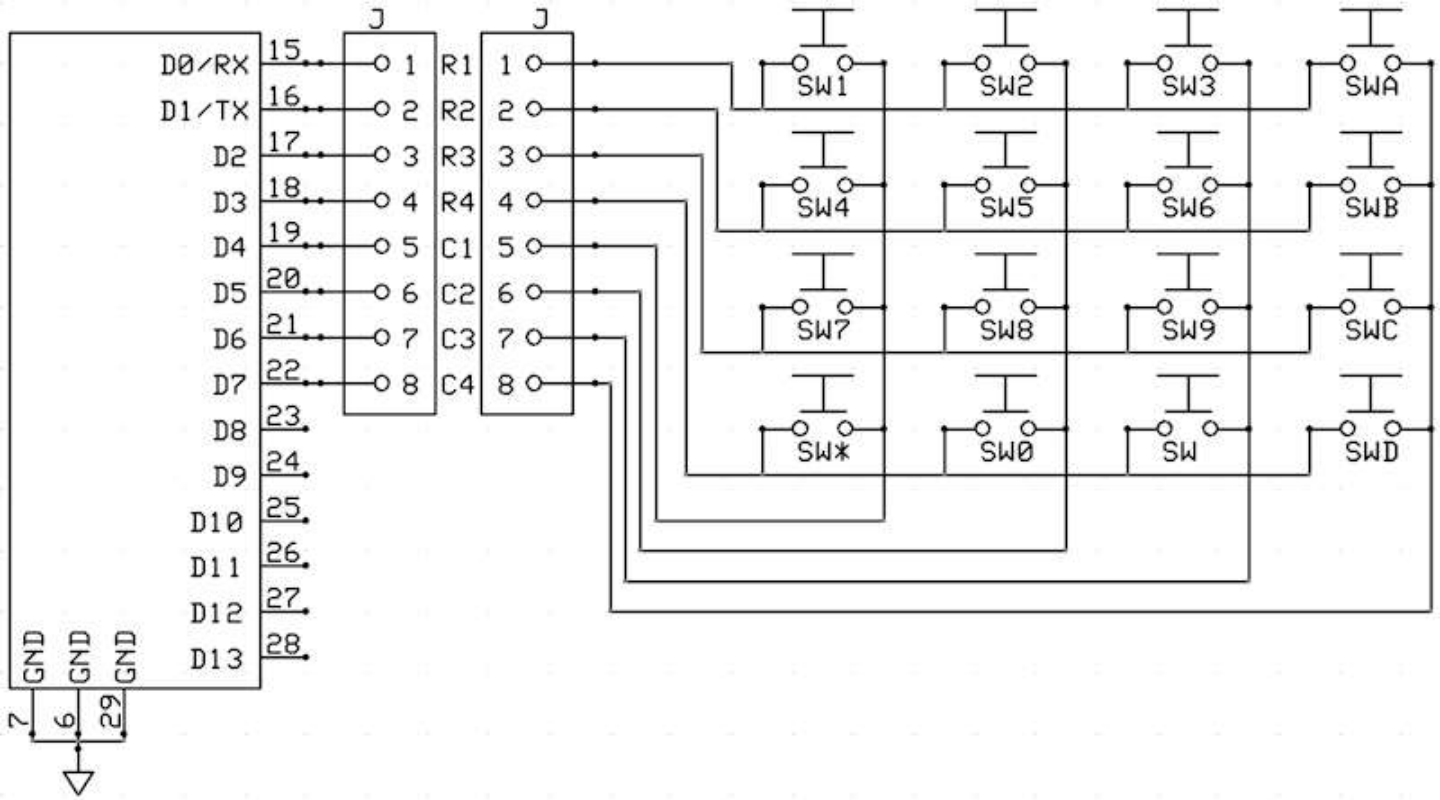


Figure 5c

## 5c1. KeyPad – Return One key

This method will scan the keypad for a single key press. It will return the encoded key if the key is pressed for 10ms. An option to this is to return the key immediately and take care of the key bounce later. This option is left to the student to implement.

The D0:D3 digital lines are used as OUTPUTs and the D4:D7 digital line are used as INPUTs with their PULLUP resistors enabled.
To SCAN, row 1 (R1) is set HIGH, then a short wait to let is stablize (not needed for mechanical switches), and the columns (C1 thru C4) are read and assembled into a four bit pattern that is shifted into a 16 bit temporary variable. This variable is tested for all ones (0xFFFF), meaning that no key was pressed.
If one or more bits are zero, then a check is made that there is only one zero since this method only return a value if only one key is pressed.
If only one zero is detected, then each time the function is called, it will a) scan the keypad a verify that the same zero is there else start over and b) if 10ms has passed the pattern didn't change, return the encoded key pattern and start testing for ALL keys to be released. Once that happens, start a new scan.

Example code is in ../ITA_Examples/Switch_5c1. TODO: Code

## 5c2. KeyPad – Return ALL keys

TODO: Flow and Code

## 6. DC Motor Control

First, a bit about PWM which is used in most motor control systems.

ref1: https://docs.arduino.cc/learn/microcontrollers/analog-output/ - used for PWM
ref2: https://learn.sparkfun.com/tutorials/pulse-width-modulation/all - basic PWM

PWM (Pulse Width Modulation) is a method for transferring energy using a modulated square wave.
How does modulation work?
If a line has a constant voltage/current on it, then 100% of the voltage/current is available. If the voltage/current on line has a 50% duty cycle (see figure 1.) then the voltage/current is only available half the time.

Figure 1

If this signal is used to light up an LED, then the LED would only be half as bright since it is getting only half the power.
The essence of PWM is to vary the duty cycle between 0% (or near 0%) to 100% (or near 100%) to control how much power is applied to the target device (i.e. LED, Motor, Heater, etc.).

Using the Arduino UNO for PWM.
The Arduino UNO (R3 and earlier), Nano, and Mini boards provide several pins (also marked with ~) for PWM generation. They are 3, 5, 6, 9, 10, and 11.
The analogWrite(pin, value) function is used to generate a PWM signal on the given pin. The value sets the percentage of the duty cycle where 0 = 0% (minimum power) and 255 = 100% (maximum power) with 128 = 50% (half power).
The default PWM frequency is 490 Hz for most PWM pins. However, pins 5 and 6 use 980 Hz.
NOTE: The lower frequency is fine for LEDs and heaters, but the higher frequency can be better for smoother motor control.

### 6.1 Simple Transistor Drive – One direction

The Arduino I/O pins can source/sink up to 16ma of current. Small toy motors can draw 50ma to several hundred milliamps. Therefore an NPN transistor is used as a current sink. A 2N2222 or 2N3904, in a TO-92 package, can handle 150ma and can be used for small toy motors. A higher current rated transistor, in a TO-220 case, would be needed for larger motors.
The transistor is a current regulating device. The more current that is applied to the Base, the more current it allows to flow from the Collector to the Emitter (for an NPN transistor).
The diode across the motor is used to shunt back voltage generated by the motor coils when the motor is turned off.
This configuration is sensitive to voltages, motor characteristics, etc. It can be used with a PWM signal, but may not be very controllable. It's best used in simple ON-OFF control schemes.

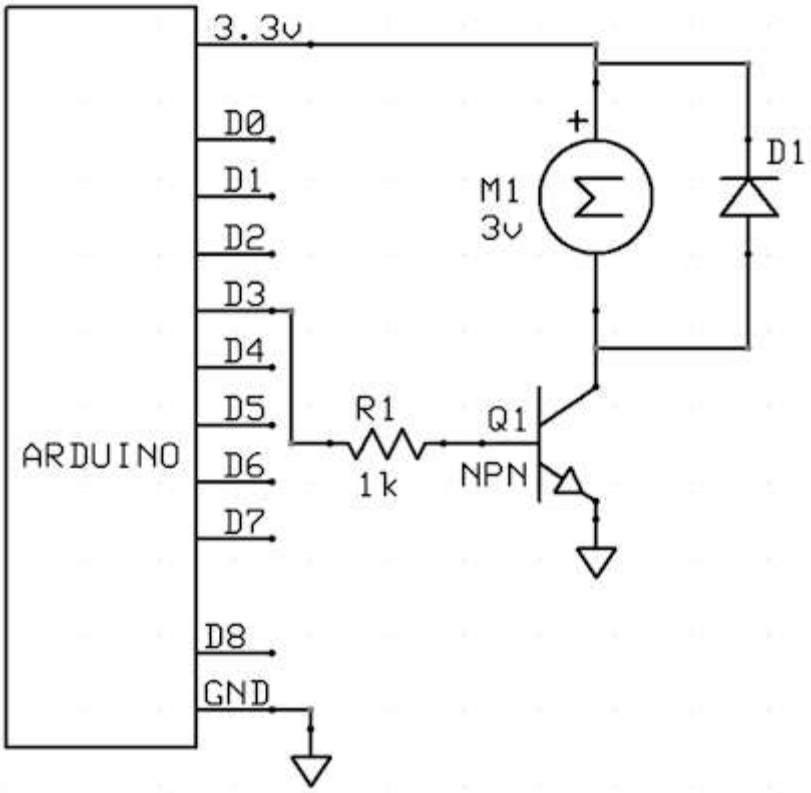


Figure 6_1

## 6.2 H-Bridge Drive – Two directions

There are several H-Bridge driver packages available. Common ones are the TB6612, the SN754410 Quad Half H-Bridge, and modules based on the L293D for higher power applications. This example uses the TB6612 dual motor driver module.

The TB6612 provides two motor sections and has 2 control lines for each section. There is also a global STBY pin that places both section in standby mode, disabling the outputs. Table 6_2 shows their functions.

| Input | | | | Output | | |
|---|---|---|---|---|---|---|
| IN1 | IN2 | PWM | STBY | OUT1 | OUT2 | Mode |
| H | H | H/L | H | L | L | Short brake |
| L | H | H | H | L | H | CCW |
| | | L | H | L | L | Short brake |
| H | L | H | H | H | L | CW |
| | | L | H | L | L | Short brake |
| L | L | H | H | OFF (High impedance) | | Stop |
| H/L | H/L | H/L | L | OFF (High impedance) | | Standby |

Table 6_2

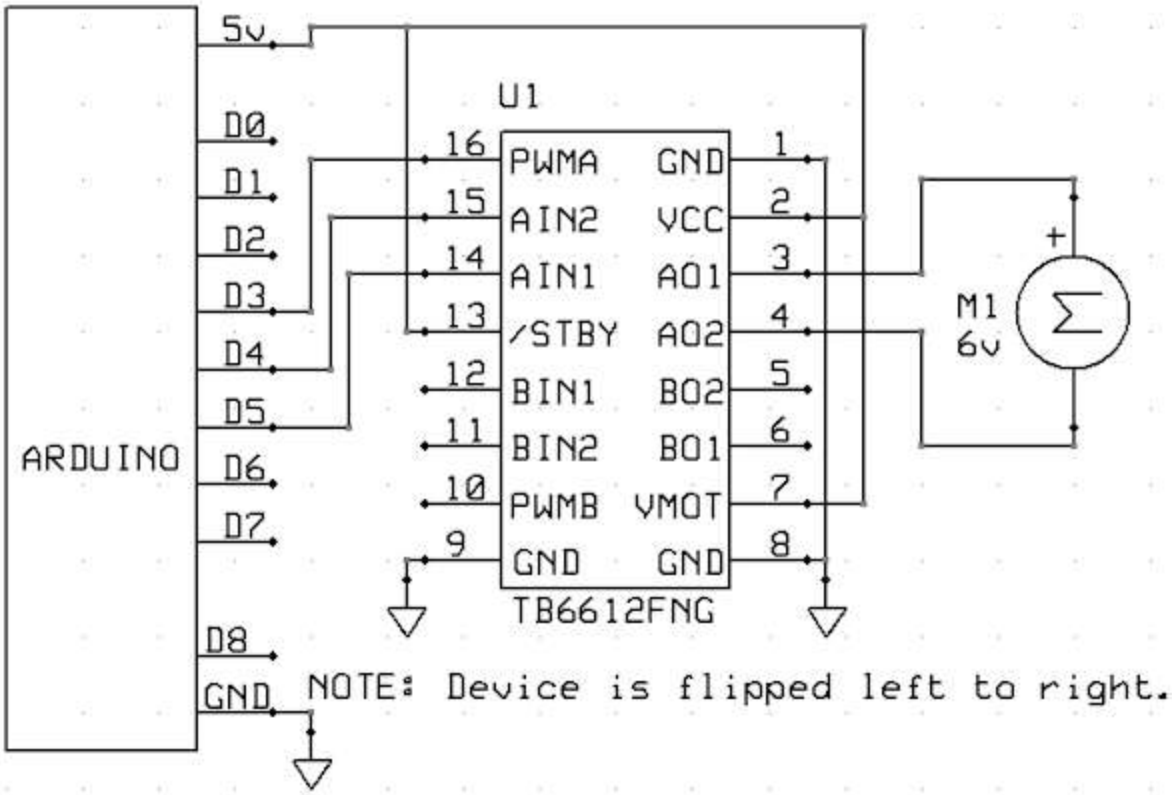


Figure 6_2

Digital I/O pin 3 will be used as a PWM signal generator by controlling it with the analogWrite(3, val) function. The val parameter will set the PWM percentage with 0:0% to 255:100%. See Motor_6a2.ino sketch in the ITA_Examples folder.

## 6.3 Quadrature Feedback